Rivanna + Cursor Remote SSH: A Verbose “For Dummies” Guide

Audience: You, setting up UVA Rivanna HPC work alongside Cursor on your laptop.
Goal: Understand what a “remote workspace” is, whether you can “bridge” it with your local dissertation project, what you must copy vs what can stay put, and how this affects your workflow (including AI assistants in Cursor).

This is **not** official UVA Research Computing documentation. For cluster policy, quotas, modules, and Slurm, use Research Computing. For SSH login details, see ssh on UVA HPC.

Related (same folder, `tenure_documents`): `GIT_MULTIPLE_MACHINES_ELEMENTARY.md`, `TENURE_STREAMLINING_AND_RESEARCH_PRIORITIES.md` (conda vs Git, research roadmap, **print checklist**).

Part 1 — Mental model: two computers, two folders

When you use **Remote SSH** in Cursor (or VS Code), you are doing this:

1. Your **Mac** runs the Cursor **window** and the editor UI.
2. The **files you open** live on **another machine** (here: a UVA HPC **login node**), at paths like `/scratch/...`, `/project/...`, or your home directory on the cluster.
3. Cursor shows that remote folder as a **workspace** — a single root directory (or a few roots if you use multi-root workspaces).

Your **Dropbox dissertation folder** on the Mac (e.g. `.../Cursor Workspace PDE/`) is a **different filesystem**. Nothing magically merges the two unless **you** connect them with git, rsync, symlinks, or a shared mount.

Short answer: Remote SSH does **not** “bridge” your local Dropbox workspace and your Rivanna folder into one unified tree by itself. You choose **which folder is the workspace** for that Cursor window: local **or** remote **or** (in some setups) **both** as separate roots in one window (see Part 5).

Part 2 — “I opened Remote SSH and got a new workspace” — is that normal?

Yes. That is expected.

- **Local Cursor:** workspace = folder on your Mac (e.g. Dropbox).
- **Remote Cursor (Remote SSH):** workspace = folder on Rivanna.

Each connection is **a separate project root** unless you explicitly add more folders. You did not lose your local project; you simply opened **another** project that happens to live on the server.

Think of it as two binders: **Binder A** (laptop) and **Binder B** (HPC). Remote SSH opens Binder B. Binder A is still on your desk.

Part 3 — Do you need to migrate the whole PDE workspace to Rivanna?

You do not *have* to move everything. What you need on Rivanna depends on what you will **run** there.

If you want to...	You typically need on Rivanna...
Run Python/scripts that read **local** pipeline outputs	Those files (or paths) on Rivanna — copy, clone, or generate there
Use the **full OpenAlex bulk** snapshot on disk	**Only** that data (already there in your job space) — your laptop does not need a second full copy
Edit the same **notebook/code** in Cursor on Rivanna	A **copy** of the repo or project files on Rivanna, or a **git clone** of the same repo
Keep **one canonical** copy of code	Best practice is **git** (see Part 6), not "only Dropbox on Mac and hope"

Common pattern (recommended):

- **Canonical code:** Git repository (private repo on GitHub/GitLab/UVA GitLab), cloned on Mac **and** on Rivanna.
- **Big data:** Lives on Rivanna only (your OpenAlex path). Code references it via **config** (environment variable or small `paths_local.py` / `paths_hpc.py` that is git-ignored).
- **Small pipeline outputs** (CSVs, JSONL samples): Either generated on Rivanna or synced selectively when needed.

You do not need to duplicate multi-terabyte OpenAlex onto your laptop for Cursor to "see" it — only the machine that **runs** the heavy job needs that path. Cursor "sees" it when your **Remote SSH workspace** is opened to a folder on Rivanna that includes or points to that data.

Part 4 — Constraints you should internalize

4.1 Login node vs compute nodes

- **Remote SSH** usually lands on a **login / front-end** node. That node is for **editing, light commands, submitting jobs**.
- **Heavy** CPU/GPU work belongs in **Slurm** (`sbatch`, `salloc`, interactive partitions), not endless Python on the login node. RC may kill abusive processes.
- "More computing power" comes from **submitting jobs** to the queue, not from the editor magically using all cluster cores in the SSH session.

4.2 Network and VPN

- Off Grounds, you typically need **UVA VPN** (e.g. UVA Anywhere) for SSH to work reliably. If SSH **hangs**, VPN is the first thing to check.

4.3 Paths differ

- On your Mac: `/Users/.../Dropbox/...`

- On Rivanna: `/home/...`, `/scratch/...`, `/project/...`, etc.
- Your scripts should avoid hard-coding one machine's paths in shared files. Use **environment variables** or machine-specific config files (git-ignored).

4.4 Scratch vs project storage

- `/scratch` is often **not backed up** and may be purged after inactivity — confirm current RC policy for your allocation.
- `/project` (or similar) may be longer-lived and shared with a group. Put things that must survive according to RC guidance.

4.5 Dropbox vs Git on the cluster

- Dropbox sync on Rivanna is **not** the default mental model. Most people use **git** + optional **rsync** for large artifacts that should not be in git.

Part 5 — Can you "bridge" workspaces? Options and pros/cons

Option A — Two Cursor windows (simplest mentally)

- **Window 1:** Local workspace (Dropbox) — daily writing, small runs, notebooks against small samples.
- **Window 2:** Remote SSH workspace on Rivanna — edit scripts that point at OpenAlex, submit Slurm jobs.

Pros: Clear separation; no accidental editing the wrong copy.
Cons: Two places to open; you must remember which file is "truth."

Option B — Git as the bridge (recommended for code)

- One **remote** Git repo; **clone** on Mac and on Rivanna (same branch, pull before work).
- Heavy data stays on Rivanna; code is small and travels via git.

Pros: Single source of truth for code; history; easy collaboration.
Cons: Learning curve if you are new to git; large binary outputs should not be committed.

Option C — Rsync / copy snapshots (occasional bridge for data)

- Periodically `rsync` project files or outputs between Mac and Rivanna (or vice versa).

Pros: Works without git; good for "push this folder to cluster once."
Cons: Easy to get **out of sync**; no merge history; you must track what you copied.

Option D — Multi-root workspace (one window, two folders)

Some editor setups allow **adding a second folder** to the workspace (local + remote is tricky; often it is **local + local** or **remote + remote** depending on product). Cursor/VS Code behavior can change between versions.

Pros: One window.
Cons: Confusing which path is which; not always available or wise for huge trees.

Practical advice: Prefer **Option A or B** until you are comfortable.

Option E — SSHFS (mount Rivanna on your Mac)

Mount a remote directory so it **looks** like a local folder.

Pros: Feels local.
Cons: Often **slow** or flaky for many small files; bad for huge datasets; editor indexing can hurt. Usually **not** recommended for OpenAlex-scale data.

Part 6 — Long-running agents (PEER, CODA, SPORT) and what you can actually lose

You described **PEER, CODA, and SPORT** as **agents you have long history with** — e.g. months in the **same** conversation with CODA. That is different from "I opened a new chat yesterday." Here is the honest distinction.

What lives where

- **Cursor** (the app on your Mac) talks to AI models over the network. The **assistant does not live inside** your Dropbox folder or inside Rivanna; it responds in **chat threads** stored/managed by the **editor product** (and whatever account sync they use).
- **Remote SSH** only changes **which folder** is the workspace for that window. It does **not** delete your other chats by itself. Your **months-long CODA thread** should still be in Cursor's chat history **if** the product still keeps it — unless you cleared history, switched accounts, or the UI puts "local workspace" and "remote workspace" threads in different sidebars (so it *feels* like you "lost" access until you open the right workspace or scroll to the old thread).

What "losing access" really means in practice

Fear	What is usually true
"Rivanna steals my agents"	**No.** Same Cursor app; different **folder** open.
"I lose the *background* we built"	**Partially real.** New chat = model starts without that thread unless you **bring context over** (paste summary, point it at files, or use project Rules). **Months of thread** = product history; if you start a brand-new chat on a remote workspace, that new chat **does not** automatically include CODA's months unless you copy/paste or summarize.
"I need the same thread forever"	Chat threads are **not** the same as **Git history**. For **reproducible** project memory, put durable facts in **files**: `README`, `docs/CONTINUITY.md`, `.cursor/rules`, decision logs. Then any agent (new thread or old) can read them.

Practical ways to protect the *substance* of long threads (not just vibes)

1. **Keep one "continuity" note in the repo** (even before Git): e.g. `docs/PROJECT_MEMORY.md` — decisions, definitions, paths, what OpenAlex slice you use, naming conventions. Point every new agent at it: "read this first."
2. **Cursor Rules** (`.cursor/rules/`) for facts you want **every** session to respect in this project — stable and versionable once you use Git.
3. **Occasionally export or summarize** marathon threads (copy the high-value bits into `PROJECT_MEMORY.md`). Products change; **files you own** last longer than any single chat log.
4. **Naming:** If CODA is "the dissertation methods thread," you can **continue that thread** when working remotely **if** Cursor still shows it — or paste a 1-page "state of the project" into the first message after switching workspaces.

Bottom line: You are not abandoning CODA by using SSH. You **might** open a **new** workspace thread that feels blank — **bridge that** with a saved summary + repo docs so the *understanding* survives even when the *chat UI* is a fresh tab.

Part 6b — Quick clarification (still true)
In Cursor, **AI chat and Composer** are features of the **Cursor application** on your machine. They are not "installed on Rivanna." When you open a **Remote SSH** workspace, you are still using Cursor; the AI can help **in that window** using files in **that** workspace.
Recommendation: Pick **one primary workflow** for code: **Git** with two clones (Mac + Rivanna) when you are ready. Use **files + Rules** so "new chat" never starts from zero on project facts.

Part 7 — Suggested workflow (concrete)

1. **Create a private Git repo** for the dissertation code (if you do not have one yet).
2. **Clone** it on your Mac (existing work: `git init` + remote, or copy files in once).
3. **Clone the same repo** on Rivanna in your project or scratch area (where your group works).
4. Add a **git-ignored** file on each machine, e.g. `local_paths.py` or use env vars:
   - `OPENALEX_BULK_ROOT=/path/on/rivanna/...`
5. **Heavy jobs:** Slurm script loads modules, `cd` to repo, runs Python with `$OPENALEX_BULK_ROOT`.
6. **Cursor:** Use **Remote SSH** when you want to edit **on Rivanna** next to data; use **local** when you want Dropbox + offline. Same repo, two clones — **git pull** between them.

Part 8 — Checklist before you "go all-in" on Rivanna

- [ ] Confirm **VPN** works from home.
- [ ] Confirm **SSH** host: `login.hpc.virginia.edu` (not deprecated hostnames).
- [ ] Know your **allocation paths** (scratch vs project) and **backup policy**.
- [ ] Know where **OpenAlex** lives (absolute path) and whether your **group** can read it.
- [ ] Decide **canonical** code home: **git** strongly preferred.
- [ ] **Do not** commit secrets (passwords, API keys). Use env vars and `.gitignore`.

Part 9 — Your answers (filled in — revise anytime)

1. **PEER, CODA, SPORT** — **Long-running Cursor agents** with **months** of shared conversation history (especially CODA). **Losing access** means losing that **thread context**, not necessarily losing Cursor itself. Mitigation: **Part 6** above + a **PROJECT_MEMORY** file in the repo.

2. **Git** — Project is currently **Dropbox-only**; you **want** to use Git. You already have a **GitHub account** (lesson plans at USMA). **What a remote is and first push/clone:** see `GIT_MULTIPLE_MACHINES_ELEMENTARY.md` in this folder (`tenure_documents`).

3. **Your own space on the HPC** — You use a `cdh` (Connected Data Hub) path with `Chas_Working` for your files and separate folders for **OpenAlex**. **Write down** absolute paths once (`pwd` in each place) for env vars; **do not** commit OpenAlex into Git.

4. **Offline / VPN** — Still open: if you often work **without VPN**, keep **local Dropbox** as the comfortable edit location and use Rivanna for **batch jobs** + Git sync.

Part 9b — Git: what "options" even are (one paragraph each)

You asked what the **options** are. At a high level:

Option	What it is
No Git (Dropbox only)	Simple, but two machines = easy **drift** ("which copy did I edit?").
Git + one remote (e.g. GitHub private)	**Canonical** code history; Mac and Rivanna **clone** the same repo; you **pull/push**. Best default for dissertation code.
Git on Rivanna only	Rare; your laptop would not have history without cloning.
Rsync instead of Git	Works for "copy folder sometimes"; worse for merging and history.

Large files: OpenAlex stays on Rivanna; **do not** commit terabytes. **Small** CSVs/JSONL might be OK if policies allow; sometimes people use **Git LFS** for medium binaries — optional later.

Part 9c — Follow-ups (updated)

1. **Git remote** — You will use **GitHub** (existing account). See `GIT_MULTIPLE_MACHINES_ELEMENTARY.md` for what "remote" means and the exact push/pull/clone dance.
2. **Extra-private data** — **None required** for repo choice right now; private repo on GitHub is still a good default habit.
3. **Paths on Rivanna** — `cdh` → `Chas_Working` + OpenAlex trees; keep paths in a small **git-ignored** env file or `PROJECT_MEMORY.md` (not the giant data itself).

Part 10 — Summary table

Question	Short answer
Can Remote SSH "merge" my Mac Dropbox folder with Rivanna automatically?	**No** — unless you use git, rsync, or open a multi-root workspace by design.
Must I copy the entire PDE folder to Rivanna?	**Only if** you want to edit/run it all there. Use **git** to avoid manual drift.
Will I lose Cursor AI on Rivanna?	**No** — Cursor still runs locally; open the **remote** folder as workspace so the AI sees those files.
Where is "more power"?	**Slurm compute nodes**, not the SSH login session itself.
Best long-term bridge for code?	**Git** + two clones (Mac + Rivanna).

References (official)

- UVA RC — SSH: https://www.rc.virginia.edu/userinfo/hpc/logintools/rivanna-ssh/
- UVA RC — Rivanna login: https://www.rc.virginia.edu/userinfo/rivanna/login/

Appendix — Research priorities (self-contained; same as `TENURE_STREAMLINING_AND_RESEARCH_PRIORITIES.md` Part 3)

After SSH / Git / Rivanna connectivity is working, the **first scientific priority** is **rough counts** to see if the research agenda is viable.

Question A — Certainty about role, place, and time: Count individuals with **high certainty** as **Assistant professor** at a **specific school** at a **specific time** (year / season / snapshot, per panel design).

Question B — Outcomes: Among those, quantify how many **promoted to Associate** (or equivalent) vs how many **left** the assistant rank without promotion (**"got out"** — define operationally: industry, other institution, etc.).

Question C — OpenAlex first: Restrict the **first** OA author disambiguation pass to **high-confidence** people who are **assistant professors**; **quantify N**. Those names feed OA matching; bulk OpenAlex on HPC follows for scale.

Question D — Performance on Rivanna: Use **downloaded OpenAlex** on the HPC for **works** and **citations** (and derived metrics) as performance measures for disambiguated authors.

Order of operations: (1) Panel / parse certainty → assistant + school + time counts. (2) High-confidence assistant subset → count + list. (3) OA disambiguation (API + HPC bulk as needed). (4) Works / citations → analysis.

Document updated with user notes on long-running agents (CODA/PEER/SPORT), Dropbox vs Git, and HPC space (`cdh` / `Chas_Working` / OpenAlex). Companion doc: `GIT_MULTIPLE_MACHINES_ELEMENTARY.md`. PDF + sun still recommended.